HOW THE SYSTEM WORKS

Cobalt Data Society Innovation Radar

An illustrated walkthrough of the data pipeline, the closed taxonomy, the three independent scores, and the Reflex app that puts it all in front of a reader.

5
LIVE DATA SOURCES

14 · 27 · 21
VERTICALS · USE CASES · TECHNOLOGIES

4
INDEPENDENT SCORES PER SPACE

216
TESTS PASSING

Prepared for Dan Ukendi

27 August 2026

CONTENTS

What's in this document

Ten sections, each built around one diagram. Read it front to back for the full mental model, or jump straight to the piece you need to explain to someone else.

> **WHAT THIS SYSTEM ACTUALLY IS**
>
> A business-facing “innovation radar.” It automatically collects institutional signals — government procurement notices, EU research grants, news — from real external sources, uses an LLM to classify each one against a closed business taxonomy, groups them into **opportunity spaces** (a Vertical × Use Case × Technology triple), and scores/explains each one for a human reader — strategist, salesperson, or presales engineer — deciding what to act on.

5,504	547	311	14 · 27 · 21	216
ARTICLES COLLECTED	CLASSIFIED	OPPORTUNITY SPACES	VERTICALS · USE CASES · TECH	TESTS PASSING

These are live counts at the time the source briefing was written (27 Aug 2026) — the local database changes every time a pipeline run happens, so treat them as a snapshot, not a constant.

SECTION 01

The big picture

Two top-level parts live in the same repository root and share one Python environment, but they talk to each other through exactly one shared surface: a SQLite file.

Everything in this system exists to answer one question well: "what should I act on, and why?" Two codebases cooperate to answer it — one gathers and scores the evidence, the other presents it.

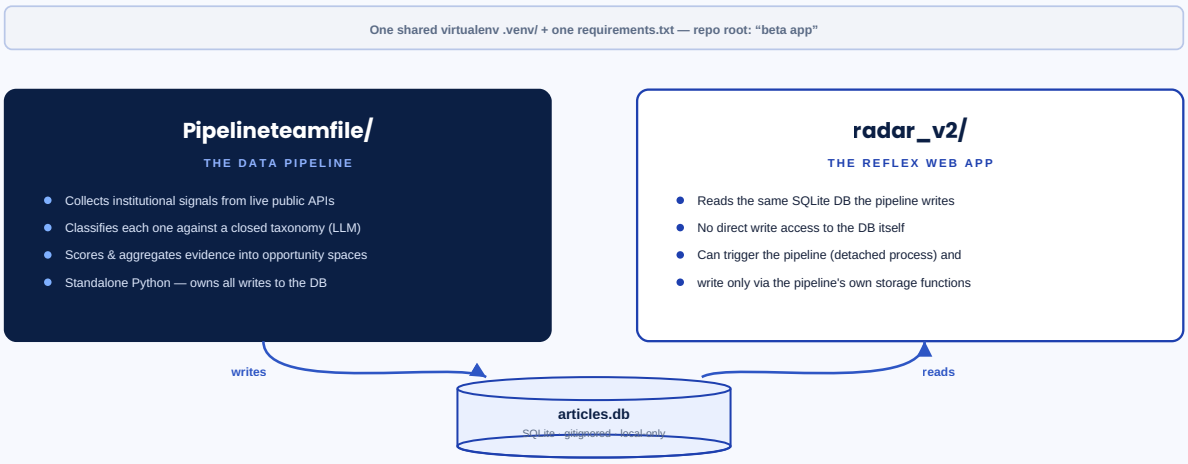

Fig. 1 — Pipelineteamfile/ owns every write to articles.db; radar_v2/ only ever reads it directly, and can trigger new writes only by calling the pipeline's own storage functions.

Why this separation matters

The data pipeline (`Pipelineteamfile/`) is standalone Python — it doesn't know or care that a web app exists. It collects evidence, classifies it, and scores it, entirely on its own schedule. The Reflex app (`radar_v2/`) is a pure consumer: every page it renders ultimately reads through one boundary module, `team_repository.py` . That boundary is what keeps the app from quietly growing its own copy of business logic that could drift out of sync with the pipeline's.

The one exception is the "Run full radar update" button in the app, which does trigger a real pipeline run — but even then, it launches the pipeline as a fully separate OS process and only ever writes to the database through the pipeline's own storage functions, never directly. That distinction — trigger vs. write — is what section 08 covers in more depth.

> **ONE THING WORTH REMEMBERING**
>
> `articles.db` is gitignored and local-only. It is **not** the same file that ends up deployed to Fly.io — the deployed copy is a frozen snapshot baked in at the last deploy, not a live sync target. Section 09 covers exactly what that implies.

SECTION 02

The data pipeline

`Pipelineteamfile/run_radar.py` orchestrates five stages, run per vertical, across four live public data sources.

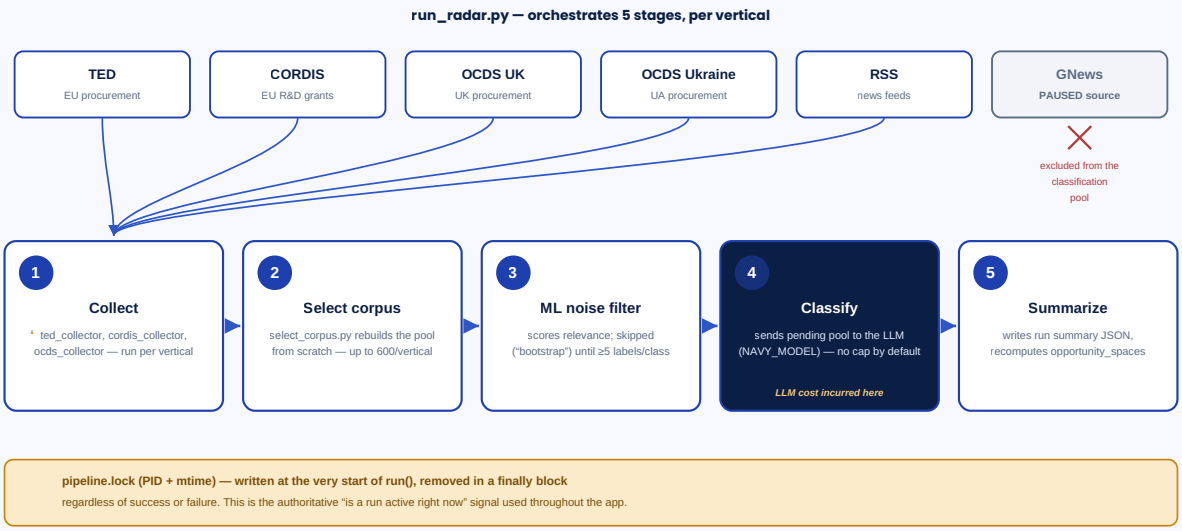

Fig. 2 — Collection sources feed a 5-stage pipeline. GNews is explicitly paused and excluded from the classification pool entirely; RSS is active.

Stage by stage

1 — Collect. `ted_collector`, `cordis_collector`, and `ocds_collector` each run once per vertical, pulling from TED (EU procurement), CORDIS (EU research/innovation grants), and OCDS UK / OCDS Ukraine (procurement) — all live, free, public APIs.

2 — Select corpus. `opportunity_classifier/collector/select_corpus.py` rebuilds the `classification_pool` table from scratch on every run — up to 600 articles per vertical, balanced across a TED-backbone / CORDIS-OCDS-fill / RSS-backfill mix, with a recent / 1-year / 5-year time spread. This is a real detail worth internalizing: **the pool is not additive.** A space's article set doesn't grow monotonically between runs just because the pool table changed — `article_classifications` itself is additive and permanent, but the pool is only ever a fresh selection mechanism.

3 — ML noise filter. `opportunity_classifier/mlfilter` scores articles for relevance before they reach the LLM. It's skipped ("bootstrap" mode) until there are at least 5 `classified` / `needs_review` labels and at least 5 `no_match` labels to train against — so on a fresh system, everything just flows through to classification.

4 — Classify. Each pending pool article goes to the LLM — currently `glm-5.1` via an OpenAI-compatible endpoint (`NAVY_MODEL` / `NAVY_BASE_URL`). As of this session, there's **no cap by default**: omitting `--limit` means the entire pending pool gets processed, which is where real token cost is incurred.

5 — Summarize. Writes a JSON run summary to `Pipelineteamfile/logs/radar_runs/{run_id}.json` and recomputes `opportunity_spaces` from whatever was just classified.

SECTION 03

The taxonomy

Everything downstream — scoring, filtering, the app's pages — is built on one closed vocabulary. The classifier is instructed to never invent an id; `null` is always a valid answer for use case or technology.

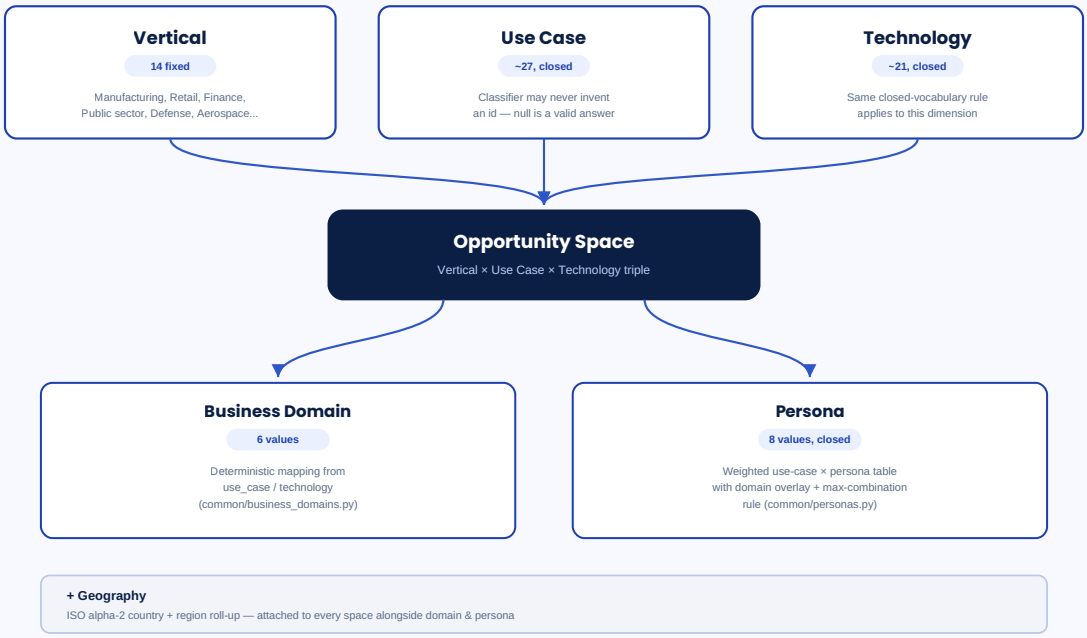

Fig. 3 — An opportunity space is a Vertical × Use Case × Technology triple. Business domain and persona are both derived from it deterministically, not chosen independently.

The three closed dimensions

14 verticals — Manufacturing, Retail, Finance/Banking/Insurance, Public/Gov, Defense, Automotive, Transportation & Construction, Lifesciences, Energy, Wholesale, Media & Entertainment, Healthcare, Natural Resources, Aerospace. Defense and Aerospace are kept as two separate verticals, not one.

~27 use cases and **~21 technologies** — closed lists with stable slug ids, configured in `opportunity_classifier/config/taxonomy.json`.

What gets derived, and how

Business domain (6 values: `ox-smart-industries`, `connectivity-solutions`, `cybersecurity`, `cloud`, `cx-customer-experience`, `ex-employee-experience`) is derived deterministically per space from its use_case/technology, via mapping tables in `common/business_domains.py` / `radar_v2/services/domains.py`. "OX: Smart Industries" is real Orange Business branding, not a typo.

Persona (8 values, closed vocabulary — no EX/HR persona) uses a weighted use-case × persona table with a domain overlay, feeding both filtering and a dampened ranking multiplier. This lives in `common/personas.py` / `radar_v2/services/personas.py`.

Geography — countries in ISO alpha-2 plus a region roll-up. Deterministic (field extraction) for TED/OCDS/CORDIS; LLM-inferred for RSS.

SECTION 04

Signal types

Six closed categories describe *why* an article is evidence of something. Where they come from — deterministic rule or LLM judgment — depends entirely on the source.

Fig. 4 — The six signal types, in tie-break priority order, and which sources get each one for free versus which ones need an LLM call.

Deterministic where possible

For TED, OCDS (UK/Ukraine), and CORDIS, the signal type is derived mechanically from the record itself — no LLM call, no cost. Feed identity alone determines TED/OCDS as `buying_signal`. CORDIS uses its own public API's project status to resolve to `proof_signal` or `tech_maturity`, depending on status and results.

For RSS, the model decides. GNews would too, if it weren't paused entirely.

Two parallel text fields

Both the deterministic path (`signal_route.py`) and the LLM path (`client.py`) populate two fields per article: `signal_type_rationale` (analyst-toned — why this type was chosen) and `signal_type_plain_summary` (a plain-language restatement of the underlying fact, for non-technical readers). The plain-summary field was added this session specifically so the app's "why hot now" explanation could show readable language instead of truncated analyst text — and as of this writing, every one of the 547 classified articles has both fields populated. Zero are missing.

> **A CAVEAT WORTH FLAGGING FORWARD**
>
> There's no quality guarantee for `signal_type_plain_summary` on a future source type that isn't yet covered by the rewritten prompt/templates. If a new source type is ever added, it needs both a rationale path and a plain-summary path (deterministic template or prompt field), or it will silently fall back to truncated rationale text.

SECTION 05

The database

One SQLite file, `Pipelineteamfile/data/articles.db` — gitignored, local-only, and not the same file that's deployed to Fly.io.

TABLE	WHAT IT HOLDS
`articles`	Raw collected records, before classification.
`article_classifications`	One row per article: taxonomy match, confidence, signal type, geography, plain summary. Additive and permanent.
`classification_pool`	The current balanced selection for classification — rebuilt every run, **not** historical.
`opportunity_spaces`	Aggregated per Vertical × Use Case × Technology triple.
`opportunity_space_domains` / `_personas` / `_regions`	Join tables attaching derived dimensions to each space.
`ml_noise_scores`	Relevance scores from the bootstrap-gated ML filter.
`sources`	Trust and audit metadata, feeding source credibility scoring.

RIGHT NOW, SPECIFICALLY

There's a large classify backlog: roughly 4,957 of the 5,504 collected articles are still unclassified, a byproduct of repeated test collection runs outpacing classification during this session. The next full pipeline run — all 14 verticals × TED+CORDIS+OCDS collection, uncapped classification of ~5k pending articles — will take real time and cost real tokens.

SECTION 06

The scoring model

`radar_v2/services/attractiveness.py` produces three independent outputs per opportunity space, plus one independent publication gate — never blended into a single number.

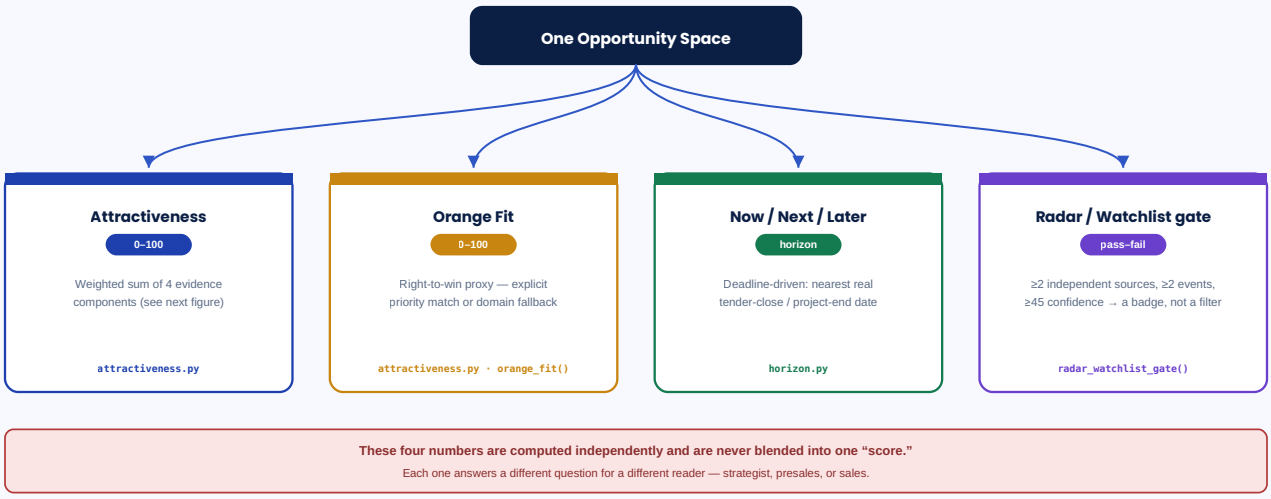

Fig. 5 — Four separate computations, none of which reads the others’ output. A fix or bias in one can never silently leak into another.

6.1 — Attractiveness score (0–100)

A weighted sum of four independently computable components. If a component is missing for a given space, it’s excluded and the remaining weights are rescaled — never counted as zero.

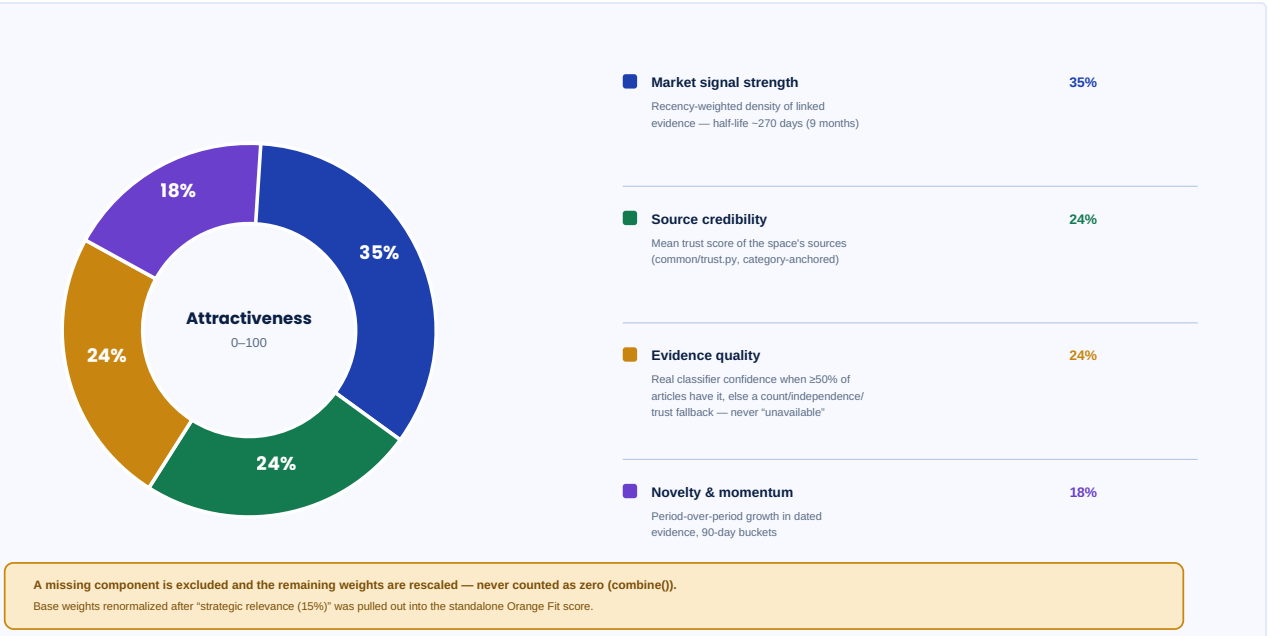

Fig. 6 — The four weighted components. Weights were renormalized this session after "strategic relevance" was pulled out into the standalone Orange Fit score below.

6.2 — Orange Fit / right-to-win proxy (0–100)

Standalone — it never enters the Attractiveness sum. Two paths, depending on whether Company → Orange priorities has anything configured yet.

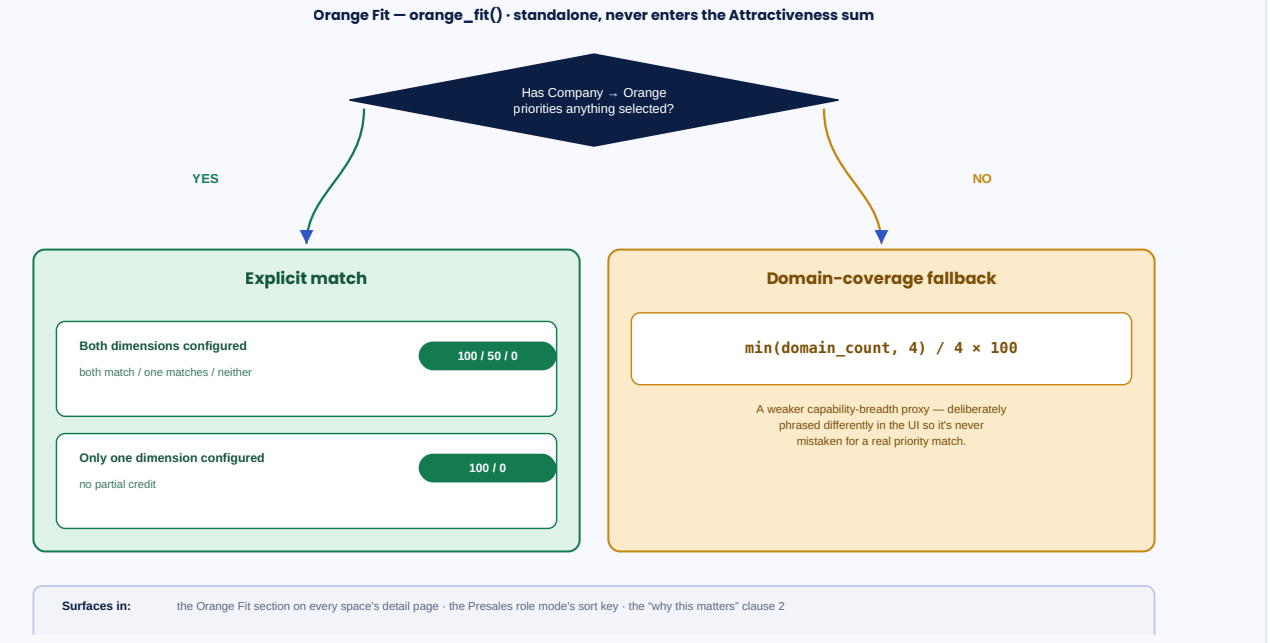

Fig. 7 — Explicit priority match when configured; a weaker domain-coverage proxy, deliberately phrased differently in the UI, while nothing is configured.

6.3 — Now / Next / Later horizon

Computed in `radar_v2/services/horizon.py`, and deliberately independent of both scores above — it reads nothing derived from `attractiveness.py`. It's deadline-driven rather than score-threshold-driven: based on the

nearest real tender-close or project-end date found in the evidence, how many signal types are converging, and how diverse the sources are.

6.4 — Radar / Watchlist publication gate

`radar_watchlist_gate()` is a pure evidence-independence check, ported from the team's own `Analysis/05_score_opportunities.py`: at least 2 independent sources, at least 2 independent events, and at least 45 confidence. It decides a badge or filter chip — it doesn't hide anything from view.

SECTION 07

Explanation fields

`radar_v2/services/explanations.py` produces three fields per space — all composed from typed clauses at render time, with no LLM call involved.

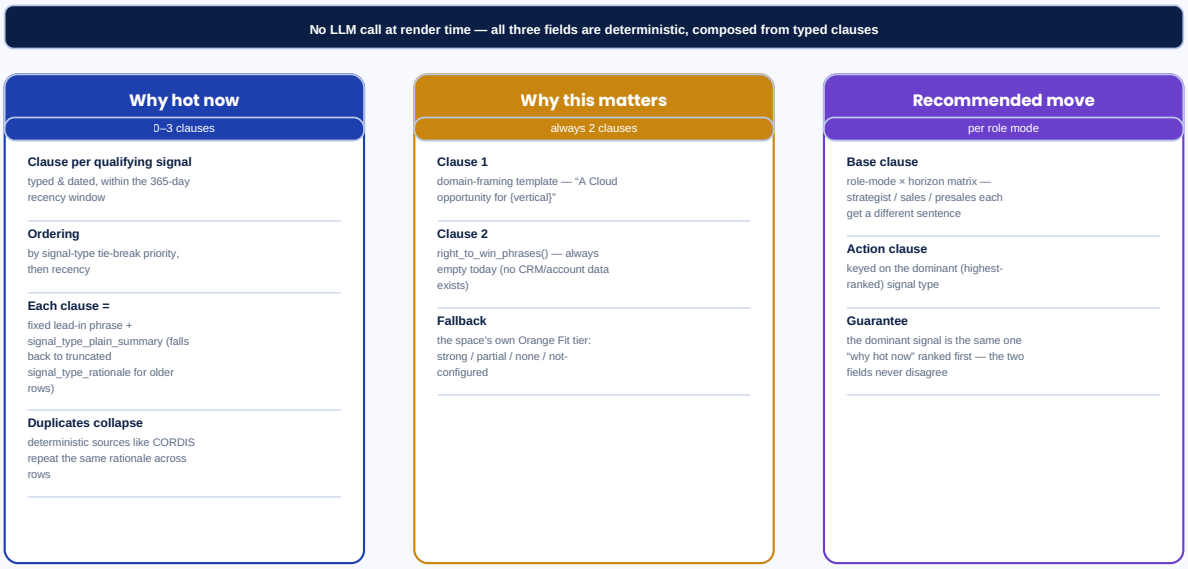

Fig. 8 — Every field is built the same way: typed clauses, deterministically selected and ordered, never generated fresh.

The design detail worth remembering is the last one: "recommended move"'s action clause is keyed on the same dominant signal type that "why hot now" ranks first. That's not a coincidence — it's a guarantee built into how both fields are composed, so the two pieces of text a reader sees on the same space can never contradict each other.

A GAP THAT'S EASY TO CONFLATE WITH SOMETHING ELSE

`right_to_win_phrases()` is always empty today — there's no CRM, account, deal, or reference-case data source anywhere in this codebase. That's a genuinely different, still-unbuilt feature from Orange Fit (which *is* real and wired). If a future request says "improve right-to-win," it's worth clarifying which of the two is actually meant.

SECTION 08

The app

A single `RadarState(rx.State)` in `radar_v2/state.py` holds essentially all UI state: opportunities list, filters, company profile, documents, discovery, reports, pipeline status, settings.

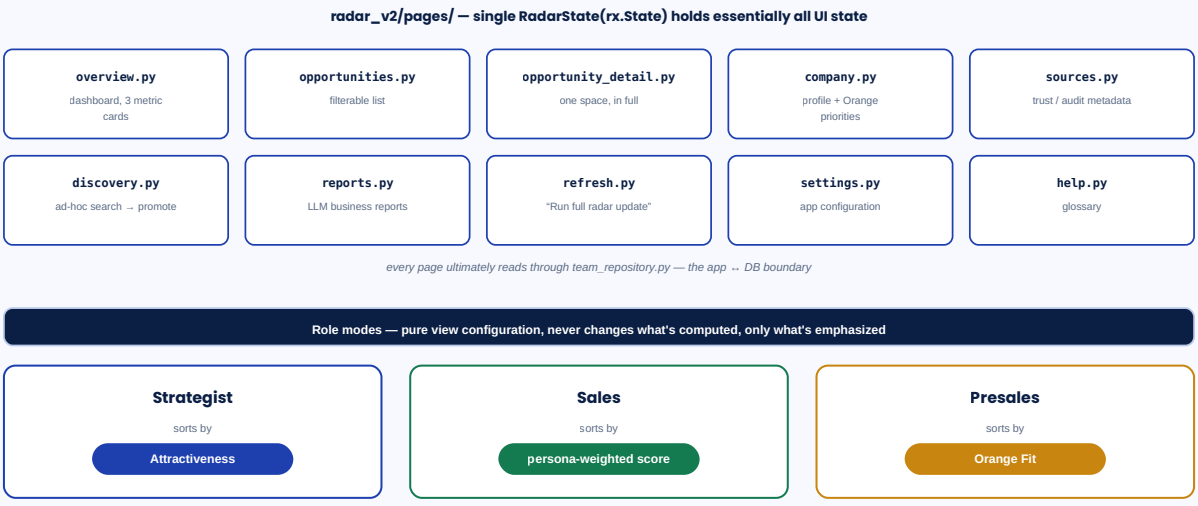

Fig. 9 — Ten pages, all reading through one boundary module. Role modes are view configuration only — they never change what's computed, only what's emphasized.

Role modes are presentation, not computation

Strategist, Sales, and Presales configure default filters, sort key, and which page regions lead, are standard, or collapse. Presales sorts by Orange Fit — the right-to-win proxy matters most to that reader. Sales sorts by the persona-weighted score. Strategist sorts by plain Attractiveness. None of this changes a single underlying number; it only changes what's in front of which reader first.

The "Run full radar update" button — three redesigns in one session

This one's worth walking through because the failure mode it fixes is subtle. The first pass made `run_pipeline` a background Reflex event with a proper async subprocess reader, which fixed an event-loop-starvation bug causing "Cannot connect to server: timeout." But a disconnect still restarted the run. So it was redesigned again: the button now launches `run_radar.py` as a fully OS-detached process (`pipeline_runner.launch_detached()`), independent of the Reflex backend process itself. A separate lightweight background task watches the lock file and log purely as an observer — losing that observer (a session drop, a page reload) never touches the run. And on any fresh page load, the app checks the lock file itself (not in-memory state) to detect an already-running detached pipeline and resume watching it, so the page never lies about "Ready" when a run is genuinely still going.

SECTION 09

Deployment

Live at **cobalt-data-society-radar.fly.dev**, region `lhr` (London). Local development and the deployed instance are two genuinely separate worlds.

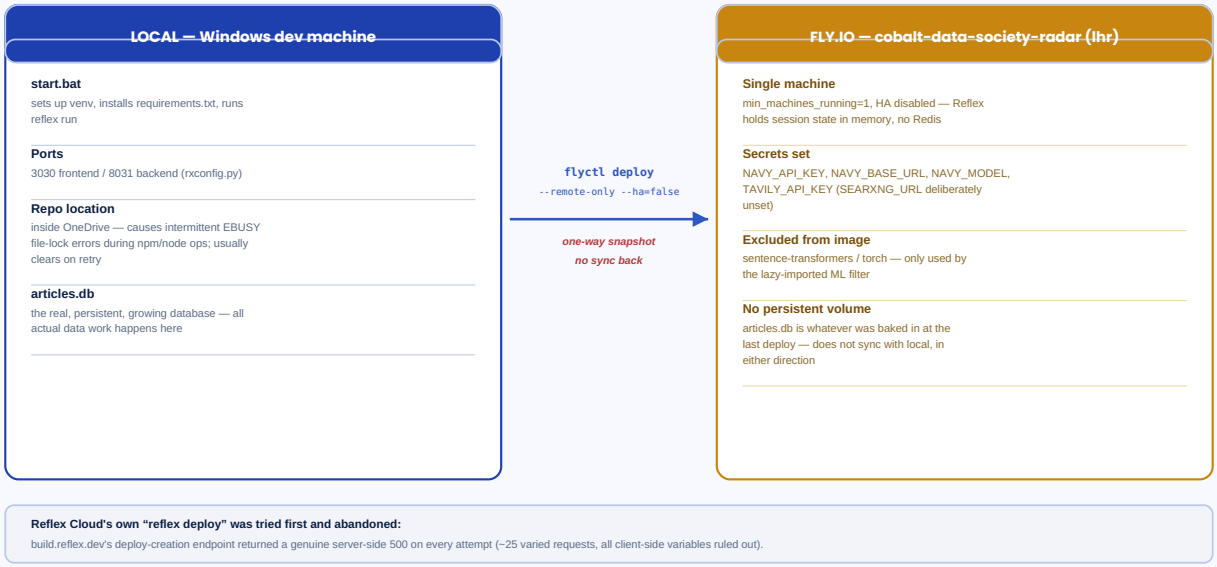

Fig. 10 — A one-way, point-in-time snapshot. Nothing that happens on Fly.io ever flows back to the local database, and nothing local automatically pushes forward either.

Why single-machine, no HA

`min_machines_running=1` with HA deliberately disabled — not an oversight. Reflex holds session state in memory with no Redis backing it, so a second machine would randomly drop sessions mid-interaction. The redeploy command matters exactly as written: `flyctl deploy --remote-only --ha=false` — omitting `--ha=false` lets Fly quietly re-add a second machine.

Why torch and sentence-transformers are excluded

They're only used by the lazy-imported ML noise-filter module, which the Reflex UI itself never touches. Excluding them from `requirements-deploy.txt` keeps the deployed image around 292 MB. If a future pipeline run on Fly ever needs that path, it will `ImportError` there specifically — local dev is unaffected either way, since nobody runs the pipeline on Fly today.

SECTION 10

Known gaps & quick reference

What to watch before assuming anything below is still current — and where to look if you need to check for yourself.

Known gaps

- **Right-to-win / CRM data genuinely does not exist.** Not the same thing as Orange Fit — see the callout in section 07 if this comes up again.
- **Large classify backlog** — ~4,957 of 5,504 collected articles are still unclassified (section 05).
- **Uncommitted local changes** as of the source briefing: `radar_v2/pages/overview.py`, `radar_v2/services/pipeline_runner.py`, `radar_v2/state.py` modified but not committed; `Dockerfile`, `.dockerignore`, `fly.toml`, `requirements-deploy.txt` untracked entirely. Check `git status` before assuming the working tree matches the last push.
- **GNews stays paused** — collected historically, excluded from new pool selection.
- `classification_pool` **is rebuilt from scratch every run**, not additive — don't assume a space's article set only grows between runs in ways tied to the pool table specifically.

Key files

CONCERN	FILE
Full pipeline orchestration	`Pipelineteamfile/run_radar.py`
LLM classification	`Pipelineteamfile/opportunity_classifier/collector/{main,client}.py`
Deterministic signal typing	`.../collector/signal_route.py`
Geography resolution	`.../collector/geo_route.py`, `common/geography.py`
Classifier prompt	`.../config/prompt_template.txt`
Taxonomy	`.../config/taxonomy.json`
DB schema/storage	`.../collector/storage.py`
Attractiveness / Orange Fit / horizon gate	`radar_v2/services/attractiveness.py`
Now/Next/Later	`radar_v2/services/horizon.py`
Explanation fields	`radar_v2/services/explanations.py`
Role modes	`radar_v2/services/role_modes.py`, `radar_v2/config/role_modes.json`
App ↔ DB read boundary	`radar_v2/services/team_repository.py`
Detached pipeline launch/status	`radar_v2/services/pipeline_runner.py`

Main state	`radar_v2/state.py`
Fly deploy config	`Dockerfile` , `fly.toml` , `.dockerignore`
Tests	`tests/` — 216 passing as of this writing

How to verify any of this yourself

- Run the test suite: `.venv\Scripts\python.exe -m pytest tests/ -q` from repo root.
- Check live local DB stats: open `Pipelineteamfile/data/articles.db` with sqlite3 directly.
- Check what's actually deployed vs. committed: `git log --oneline -5` , `git status` , and compare against the live Fly.io URL.

> **THE ONE RULE THAT MATTERS MORE THAN ANY NUMBER IN THIS DOCUMENT**
>
> Don't trust any specific figure here as current without re-checking. The database in particular changes every time a pipeline run happens — this document explains how the system works, not what its numbers are today.